Architecture 1001:
x86-64 Assembly
Mystery Listery 1 Solved!

Xeno Kovah – 2021
xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

Mystery Listery

- Why do the GCC/Clang HelloWorlds have balanced Push/Pop instructions but Visual Studio doesn't?
- What's up with the sub/add 0x28 in main()?
- **Why is VS over-allocating space for a single local variable?**

Mystery Listery

- Why do the GCC/Clang HelloWorlds have balanced Push/Pop instructions but Visual Studio doesn't?
- What's up with the sub/add 0x28 in main()?
- **Why is VS over-allocating space for a single local variable?**

SingleLocalVariable.c

Adding a single local variable

```
int func(){
    int i = 0x5ca1ab1e;
    return i;
}
int main(){
    return func();
}
```

```
func:
0000000140001000  sub           rsp,18h
0000000140001004  mov           dword ptr [rsp],5CA1AB1Eh
000000014000100B  mov           eax,dword ptr [rsp]
000000014000100E  add           rsp,18h
0000000140001012  ret
main:
0000000140001020  sub           rsp,28h
0000000140001024  call          func (0140001000h)
0000000140001029  add           rsp,28h
000000014000102D  ret
```

SingleLocalVariable2.c

Functions with a single local variable

```
int func3() {
    int i = 0x7a11;
    return i;
}
int func2(){
    int j = 0x7a1e;
    return func3();
}
int func(){
    return func2();
}
int main(){
    return func();
}
```

SingleLocalVariable2.c

Functions with a single local variable

```
int func3() {
    int i = 0x7a11;
    return i;
}
int func2(){
    int j = 0x7a1e;
    return func3();
}
int func(){
    return func2();
}
int main(){
    return func();
}
```

```
func3:
0000000140001000  sub          rsp,18h
0000000140001004  mov          dword ptr [rsp],07A11h
000000014000100B  mov          eax,dword ptr [rsp]
000000014000100E  add          rsp,18h
0000000140001012  ret
func2:
0000000140001020  sub          rsp,38h
0000000140001024  mov          dword ptr [rsp+20h],7A1Eh
000000014000102C  call         0000000140001000
0000000140001031  add          rsp,38h
0000000140001035  ret
func:
0000000140001040  sub          rsp,28h
0000000140001044  call         0000000140001020
0000000140001049  add          rsp,28h
000000014000104D  ret
main:
0000000140001060  sub          rsp,28h
0000000140001064  call         0000000140001040
0000000140001069  add          rsp,28h
000000014000106D  ret
```

SingleLocalVariable2.c

Functions with a single local variable

```
int func3() {
    int i = 0x7a11;
    return i;
}
int func2(){
    int j = 0x7a1e;
    return func3();
}
int func(){
    return func2();
}
int main(){
    return func();
}
```




```
func3:
0000000140001000  sub          rsp,18h
0000000140001004  mov          dword ptr [rsp],07A11h
000000014000100B  mov          eax,dword ptr [rsp]
000000014000100E  add          rsp,18h
0000000140001012  ret
func2:
0000000140001020  sub          rsp,38h
0000000140001024  mov          dword ptr [rsp+20h],7A1Eh
000000014000102C  call         0000000140001000
0000000140001031  add          rsp,38h
0000000140001035  ret
func:
0000000140001040  sub          rsp,28h
0000000140001044  call         0000000140001020
0000000140001049  add          rsp,28h
000000014000104D  ret
main:
0000000140001060  sub          rsp,28h
0000000140001064  call         0000000140001040
0000000140001069  add          rsp,28h
000000014000106D  ret
```

SingleLocalVariable2.c

Functions with a single local variable

```
int func3() {
    int i = 0x7a11;
    return i;
}
int func2(){
    int j = 0x7a1e;
    return func3();
}
int func(){
    return func2();
}
int main(){
    return func();
}
```

```
func3:
0000000140001000  sub          rsp,18h
0000000140001004  mov          dword ptr [rsp],07A11h
000000014000100B  mov          eax,dword ptr [rsp]
000000014000100E  add          rsp,18h
0000000140001012  ret
func2:
0000000140001020  sub          rsp,38h
0000000140001024  mov          dword ptr [rsp+20h],7A1Eh
000000014000102C  call         0000000140001000
0000000140001031  add          rsp,38h
0000000140001035  ret
func:
0000000140001040  sub          rsp,28h
0000000140001044  call         0000000140001020
0000000140001049  add          rsp,28h
000000014000104D  ret
main:
0000000140001060  sub          rsp,28h
0000000140001064  call         0000000140001040
0000000140001069  add          rsp,28h
000000014000106D  ret
```



STOP

&&

STEP THROUGH THE ASSEMBLY

&&

DRAW A STACK DIAGRAM!



STOP

&&

STEP THROUGH THE ASSEMBLY

&&

DRAW A STACK DIAGRAM!

00000000`0014FE08	return address = 00000001`40001389

main()
frame

00000000`0014FE08	return address = 00000001`40001389
00000000`0014FE00	undef
00000000`0014FDF8	undef
00000000`0014FDF0	undef
00000000`0014FDE8	undef
00000000`0014FDE0	undef

main()
frame

00000000`0014FE08	return address = 00000001`40001389
00000000`0014FE00	undef
00000000`0014FDF8	undef
00000000`0014FDF0	undef
00000000`0014FDE8	undef
00000000`0014FDE0	undef
00000000`0014FDD8	return address = 00000001`40001069

	00000000`0014FE08	return address = 00000001`40001389
main() frame	00000000`0014FE00	undef
	00000000`0014FDF8	undef
	00000000`0014FDF0	undef
	00000000`0014FDE8	undef
	00000000`0014FDE0	undef
	00000000`0014FDD8	return address = 00000001`40001069
func() frame	00000000`0014FDD0	undef
	00000000`0014FDC8	undef
	00000000`0014FDC0	undef
	00000000`0014FDB8	undef
	00000000`0014FDB0	undef

	00000000`0014FE08	return address = 00000001`40001389
main() frame	00000000`0014FE00	undef
	00000000`0014FDF8	undef
	00000000`0014FDF0	undef
	00000000`0014FDE8	undef
	00000000`0014FDE0	undef
	00000000`0014FDD8	return address = 00000001`40001069
func() frame	00000000`0014FDD0	undef
	00000000`0014FDC8	undef
	00000000`0014FDC0	undef
	00000000`0014FDB8	undef
	00000000`0014FDB0	undef
	00000000`0014FDA8	return address = 00000001`40001049

main() frame

func() frame

func2() frame

00000000`0014FE08	return address = 00000001`40001389
00000000`0014FE00	undef
00000000`0014FDF8	undef
00000000`0014FDF0	undef
00000000`0014FDE8	undef
00000000`0014FDE0	undef
00000000`0014FDD8	return address = 00000001`40001069
00000000`0014FDD0	undef
00000000`0014FDC8	undef
00000000`0014FDC0	undef
00000000`0014FDB8	undef
00000000`0014FDB0	undef
00000000`0014FDA8	return address = 00000001`40001049
00000000`0014FDA0	undef
00000000`0014FD98	undef
00000000`0014FD90	undef`00007a1e
00000000`0014FD88	undef
00000000`0014FD80	undef
00000000`0014FD78	undef
00000000`0014FD70	undef

	00000000`0014FE08	return address = 00000001`40001389
main() frame	00000000`0014FE00	undef
	00000000`0014FDF8	undef
	00000000`0014FDF0	undef
	00000000`0014FDE8	undef
	00000000`0014FDE0	undef
	00000000`0014FDD8	return address = 00000001`40001069
func() frame	00000000`0014FDD0	undef
	00000000`0014FDC8	undef
	00000000`0014FDC0	undef
	00000000`0014FDB8	undef
	00000000`0014FDB0	undef
	00000000`0014FDA8	return address = 00000001`40001049
func2() frame	00000000`0014FDA0	undef
	00000000`0014FD98	undef
	00000000`0014FD90	undef`00007a1e
	00000000`0014FD88	undef
	00000000`0014FD80	undef
	00000000`0014FD78	undef
	00000000`0014FD70	undef
	00000000`0014FD68	return address = 00000001`40001031

	00000000`0014FE08	return address = 00000001`40001389
main() frame	00000000`0014FE00	undef
	00000000`0014FDF8	undef
	00000000`0014FDF0	undef
	00000000`0014FDE8	undef
	00000000`0014FDE0	undef
	00000000`0014FDD8	return address = 00000001`40001069
func() frame	00000000`0014FDD0	undef
	00000000`0014FDC8	undef
	00000000`0014FDC0	undef
	00000000`0014FDB8	undef
	00000000`0014FDB0	undef
	00000000`0014FDA8	return address = 00000001`40001049
func2() frame	00000000`0014FDA0	undef
	00000000`0014FD98	undef
	00000000`0014FD90	undef`00007a1e
	00000000`0014FD88	undef
	00000000`0014FD80	undef
	00000000`0014FD78	undef
	00000000`0014FD70	undef
	00000000`0014FD68	return address = 00000001`40001031

main() frame	00000000`0014FDF8	undef
	00000000`0014FDF0	undef
	00000000`0014FDE8	undef
	00000000`0014FDE0	undef
	00000000`0014FDD8	return address = 00000001`40001069
	00000000`0014FDD0	undef
func() frame	00000000`0014FDC8	undef
	00000000`0014FDC0	undef
	00000000`0014FDB8	undef
	00000000`0014FDB0	undef
	00000000`0014FDA8	return address = 00000001`40001049
	00000000`0014FDA0	undef
	00000000`0014FD98	undef
	00000000`0014FD90	undef`00007a1e
func2() frame	00000000`0014FD88	undef
	00000000`0014FD80	undef
	00000000`0014FD78	undef
	00000000`0014FD70	undef
	00000000`0014FD68	return address = 00000001`40001031
	00000000`0014FD60	undef
func3() frame	00000000`0014FD58	undef
	00000000`0014FD50	undef`00007a11

"The stack will always be maintained 16-byte aligned, except within the prolog (for example, after the return address is pushed) ..." [1]

I left out further exceptions which you can go read on your own if you'd like

[1] “Stack usage” reference https://docs.microsoft.com/en-us/cpp/build/stack-usage?view=msvc-160

main() frame	00000000`0014FE00	undef
	00000000`0014FDF8	undef
	00000000`0014FDF0	undef
	00000000`0014FDE8	undef
	00000000`0014FDE0	undef
	00000000`0014FDD8	return address = 00000001`40001069
func() frame	00000000`0014FDD0	undef
	00000000`0014FDC8	undef
	00000000`0014FDC0	undef
	00000000`0014FDB8	undef
	00000000`0014FDB0	undef
	00000000`0014FDA8	return address = 00000001`40001049
func2() frame	00000000`0014FDA0	undef
	00000000`0014FD98	undef
	00000000`0014FD90	undef`00007a1e
	00000000`0014FD88	undef
	00000000`0014FD80	undef
	00000000`0014FD78	undef
	00000000`0014FD70	undef
	00000000`0014FD68	return address = 00000001`40001031
func3() frame	00000000`0014FD60	undef
	00000000`0014FD58	undef
	00000000`0014FD50	undef`00007a11

main() frame	00000000`0014FE00	*16-byte-stack-alignment padding*
	00000000`0014FDF8	undef
	00000000`0014FDF0	undef
	00000000`0014FDE8	undef
	00000000`0014FDE0	undef
	00000000`0014FDD8	return address = 00000001`40001069
	00000000`0014FDD0	*16-byte-stack-alignment padding*
func() frame	00000000`0014FDC8	undef
	00000000`0014FDC0	undef
	00000000`0014FDB8	undef
	00000000`0014FDB0	undef
	00000000`0014FDA8	return address = 00000001`40001049
	00000000`0014FDA0	*16-byte-stack-alignment padding*
func2() frame	00000000`0014FD98	undef
	00000000`0014FD90	undef`00007a1e
	00000000`0014FD88	undef
	00000000`0014FD80	undef
	00000000`0014FD78	undef
	00000000`0014FD70	undef
	00000000`0014FD68	return address = 00000001`40001031
func3() frame	00000000`0014FD60	*16-byte-stack-alignment padding*
	00000000`0014FD58	undef
	00000000`0014FD50	undef`00007a11

main() frame	00000000`0014FE00	*16-byte-stack-alignment padding*
	00000000`0014FDF8	undef
	00000000`0014FDF0	undef
	00000000`0014FDE8	undef
	00000000`0014FDE0	undef
	00000000`0014FDD8	return address = 00000001`40001069
	00000000`0014FDD0	*16-byte-stack-alignment padding*
func() frame	00000000`0014FDC8	undef
	00000000`0014FDC0	undef
	00000000`0014FDB8	undef
	00000000`0014FDB0	undef
	00000000`0014FDA8	return address = 00000001`40001049
	00000000`0014FDA0	*16-byte-stack-alignment padding*
func2() frame	00000000`0014FD98	undef
	00000000`0014FD90	undef`00007a1e
	00000000`0014FD88	undef
	00000000`0014FD80	undef
	00000000`0014FD78	undef
	00000000`0014FD70	undef
	00000000`0014FD68	return address = 00000001`40001031
func3() frame	00000000`0014FD60	*16-byte-stack-alignment padding*
	00000000`0014FD58	undef
	00000000`0014FD50	undef`00007a11

main() frame	00000000`0014FE00	*16-byte-stack-alignment padding*
	00000000`0014FDF8	undef
	00000000`0014FDF0	undef
	00000000`0014FDE8	undef
	00000000`0014FDE0	undef
	00000000`0014FDD8	return address = 00000001`40001069
	00000000`0014FDD0	*16-byte-stack-alignment padding*
func() frame	00000000`0014FDC8	undef
	00000000`0014FDC0	undef
	00000000`0014FDB8	undef
	00000000`0014FDB0	undef
	00000000`0014FDA8	return address = 00000001`40001049
	00000000`0014FDA0	*16-byte-stack-alignment padding*
func2() frame	00000000`0014FD98	*16-byte-stack-alignment padding*
	00000000`0014FD90	undef`00007a1e
	00000000`0014FD88	undef
	00000000`0014FD80	undef
	00000000`0014FD78	undef
	00000000`0014FD70	undef
	00000000`0014FD68	return address = 00000001`40001031
func3() frame	00000000`0014FD60	*16-byte-stack-alignment padding*
	00000000`0014FD58	*16-byte-stack-alignment padding*
	00000000`0014FD50	undef`00007a11


Why would you say something so controversial yet so brave?

DoubleLocalVariable.c

```c
int func() {
    long long i = 0xf01dab1ef007ba11;
    long long j = 0x0b57ac1e5;
    return i+j;
}
int main() {
    return func();
}
```

```
func:
0000000140001000  sub          rsp,18h
0000000140001004  mov          rax,0F01DAB1EF007BA11h
000000014000100E  mov          qword ptr [rsp+8],rax
0000000140001013  mov          eax,0B57AC1E5h
0000000140001018  mov          qword ptr [rsp],rax
000000014000101C  mov          rax,qword ptr [rsp]
0000000140001020  mov          rcx,qword ptr [rsp+8]
0000000140001025  add          rcx,rax
0000000140001028  mov          rax,rcx
000000014000102B  add          rsp,18h
000000014000102F  ret
main:
0000000140001060  sub          rsp,28h
0000000140001064  call         0000000140001040
0000000140001069  add          rsp,28h
000000014000106D  ret
```



STOP

&&

STEP THROUGH THE ASSEMBLY

&&

DRAW A STACK DIAGRAM!



STOP

&&

STEP THROUGH THE ASSEMBLY

&&

DRAW A STACK DIAGRAM!

	00000000`0014FE08	return address = 00000001`40001369
main() frame	00000000`0014FE00	*16-byte-stack-alignment padding*
	00000000`0014FDF8	undef
	00000000`0014FDF0	undef
	00000000`0014FDE8	undef
	00000000`0014FDE0	undef
	00000000`0014FDD8	return address = 00000001`40001069
func() frame	00000000`0014FDD0	*16-byte-stack-alignment padding*
	00000000`0014FDC8	0xF01DAB1E1F007BA11
	00000000`0014FDC0	0x0000000`B57AC1E5h

TripleLocalVariable.c

Adding a single local variable

```
int func() {
    long long i = 0xf01dab1ef007ba11;
    long long j = 0x0b57ac1e5;
    long long k = 0x57abbadabad00;
    return i+j;
}
int main() {
    return func();
}
```

```
func:
0000000140001000  sub          rsp,28h
0000000140001004  mov          rax,0F01DAB1EF007BA11h
000000014000100E  mov          qword ptr [rsp+8],rax
0000000140001013  mov          eax,0B57AC1E5h
0000000140001018  mov          qword ptr [rsp],rax
000000014000101C  mov          rax,57ABBADABAD00h
0000000140001026  mov          qword ptr [rsp+10h],rax
000000014000102B  mov          rax,qword ptr [rsp]
000000014000102F  mov          rcx,qword ptr [rsp+8]
0000000140001034  add          rcx,rax
0000000140001037  mov          rax,rcx
000000014000103A  add          rsp,28h
000000014000103E  ret
main:
0000000140001050  sub          rsp,28h
0000000140001054  call         0000000140001000
0000000140001059  add          rsp,28h
000000014000105D  ret
```



STOP

&&

STEP THROUGH THE ASSEMBLY

&&

DRAW A STACK DIAGRAM!



STOP

&&

STEP THROUGH THE ASSEMBLY

&&

DRAW A STACK DIAGRAM!

	00000000`0014FE08	return address = 00000001`40001369
main() frame	00000000`0014FE00	*16-byte-stack-alignment padding*
	00000000`0014FDF8	undef
	00000000`0014FDF0	undef
	00000000`0014FDE8	undef
	00000000`0014FDE0	undef
	00000000`0014FDD8	return address = 00000001`40001069
func() frame	00000000`0014FDD0	*16-byte-stack-alignment padding*
	00000000`0014FDC8	*16-byte-stack-alignment padding*
	00000000`0014FDC0	0x57ABBADABAD00
	00000000`0014FDB8	0xF01DAB1E1F007BA11
	00000000`0014FDB0	0x0000000`B57AC1E5h

TOO-LARGE LOCAL VARIABLE ALLOCATIONS





16-BYTE-ALIGNMENT
PADDING

Mystery Listery

- Why do the GCC/Clang HelloWorlds have balanced Push/Pop instructions but Visual Studio doesn't?
- What's up with the sub/add 0x28 in main()?
- **Why is VS over-allocating space for a single local variable?**

Mystery Listery

- Why do the GCC/Clang HelloWorlds have balanced Push/Pop instructions but Visual Studio doesn't?
- What's up with the sub/add 0x28 in main()?
- **Why is VS over allocating space for a single local variable?**



